

Yes. And actually, **now that I know this is a showcase/portfolio project**, I would simplify the architecture **a lot**.

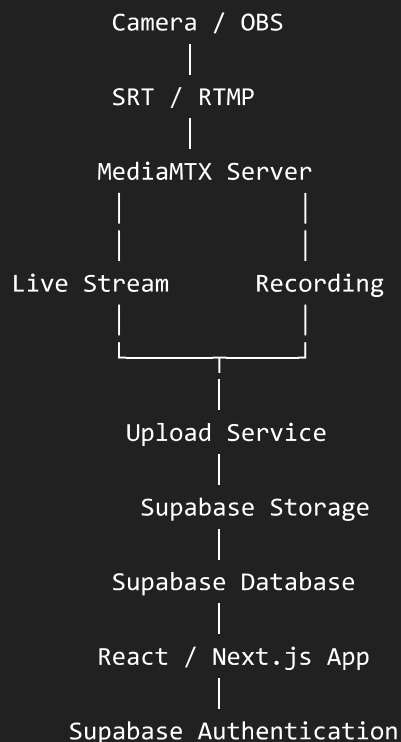
The previous architecture I suggested is closer to something you'd deploy for a real company. For a university/final-year/showcase project, that's honestly overkill.

Your goal should be:

"Build something that demonstrates the complete workflow while keeping the infrastructure easy to understand."

That's exactly what interviewers and project evaluators care about.

I would use something like this



That's literally it.

Only **three main pieces**.

- Media Server
- Web App
- Supabase

Very clean.

Could Supabase replace lots of things?

Absolutely.

Instead of having

- PostgreSQL
- Auth service
- Storage
- API

You simply get

- ☒ PostgreSQL
- ☒ Authentication
- ☒ Object Storage
- ☒ Realtime
- ☒ SDK
- ☒ Admin Dashboard

all inside one platform.

For a project...

It's honestly fantastic.

What each part does

1. MediaMTX

This ONLY handles video.

Receive stream

↓

Broadcast

↓

Record

Nothing else.

2. Supabase

Handles

Users

Streams

Recordings

Storage

Authentication

Basically your backend database.

Example

Users Table

Streams Table

Recordings Table

Video files go into

Storage Bucket

recordings/

stream1.mp4

stream2.mp4

3. Web App

The dashboard.

It does things like

Login

↓

Create Stream

↓

Start Stream

↓

Watch Live

↓

Previous Recordings



Delete Recording

Very simple.

What about the backend?

Here's the nice part.

You might not even need a separate backend server.

Your web app can directly talk to Supabase.

Browser



Supabase SDK



Database



Storage



Authentication

For project scale,

that's perfectly acceptable.

If you need to do something sensitive (like generating signed upload URLs or controlling MediaMTX), you can use a few API routes in your web app.

Recording Upload

This is probably the part you're wondering about.

After MediaMTX finishes recording:

stream001.mp4

A tiny uploader script watches the recordings folder.

```
New file appears
↓
Upload to Supabase Storage
↓
Save metadata in database
↓
Done
```

That's maybe 100–200 lines of code.

Very manageable.

Monitoring

You don't need Prometheus and Grafana.

Just build your own dashboard.

For example:

```
● Live
Bitrate
Duration
Current Viewers
Recording Status
Upload Progress
```

You can get much of this from MediaMTX's API and show it in your own UI.

It actually looks more impressive in a demo because it's **your** dashboard.

Authentication

Don't build it.

Just use Supabase Auth.

Login

↓

JWT

↓

Dashboard

Done.

Storage

Use Supabase Storage.

recordings/

event001.mp4

event002.mp4

event003.mp4

It already has:

- Upload
 - Delete
 - Public URLs
 - Private buckets
 - Signed URLs
-

Database

Maybe only four tables.

Users

Streams

Recordings

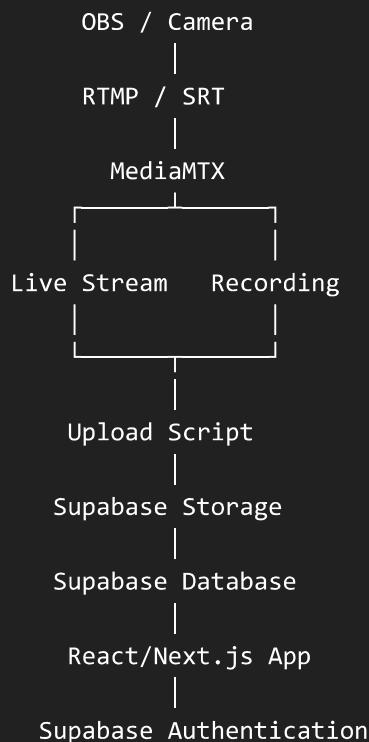
Devices

That's enough.

Notifications

Supabase Realtime can notify your dashboard instantly when a stream starts, stops, or a recording finishes uploading.

Final architecture I'd recommend



My recommendation for a showcase project

If I were building this as a portfolio project, I'd use:

- **MediaMTX** → Streaming server
- **FFmpeg** → Only if you need transcoding, thumbnails, or MP4 conversion (otherwise let MediaMTX do the recording)
- **Supabase** → Authentication, PostgreSQL database, Storage, and Realtime
- **React or Next.js** → Dashboard and management interface
- **One VPS** → Runs MediaMTX (and FFmpeg if needed)
- **One Supabase project** → Everything else

That gives you a system that's **easy to explain**, **easy to deploy**, and still demonstrates real-world concepts like live streaming, cloud recording, authentication, metadata management, and remote viewing—without the complexity of managing half a dozen infrastructure services.

I actually think this simpler architecture makes for a **better showcase** because people reviewing your project can understand it in a few minutes instead of trying to follow a

production-scale distributed system.

okay now tell me finalized services for each of these components

- live streaming component
- cloud storage
- recording and encoding
- dashboard / monitoering
- failover control
- authentication system

Given your goal (showcase project, lowest cost, easiest possible setup, but still looks professional), this is what I would finalize.

Component	Service/Technology	Why this choice
	MediaMTX	
	Supabase Storage	
	MediaMTX + FFmpeg	
	Your own Web Dashboard + MediaMTX API	
	Simple Auto-Reconnect + Docker Restart Policies	
	Supabase Auth	

Overall stack

Frontend

React / Next.js

Backend

Minimal Node.js API (only if needed)

Streaming

MediaMTX

Recording

MediaMTX + FFmpeg

Storage

Supabase Storage

Database

Supabase PostgreSQL

Authentication

Supabase Auth

Monitoring

Custom Dashboard + MediaMTX API

Deployment

Docker Compose

Why I picked these

Live Streaming → MediaMTX

This is the heart of your project.

It can:

- Receive RTMP/SRT
- Convert to HLS/WebRTC
- Serve viewers
- Record streams
- Provide APIs for status

One application replaces what would otherwise require multiple services.

Cloud Storage → Supabase Storage

Instead of:

- AWS S3
- IAM
- Buckets
- Policies
- Credentials

You simply:

```
recordings/  
event001.mp4
```

and upload through the Supabase SDK.

Very beginner-friendly.

Recording → MediaMTX + FFmpeg

MediaMTX records automatically.

FFmpeg is only used when you want features like:

- Different resolutions
- Watermarks
- Thumbnail generation
- MP4 conversion
- Video clipping

If your project only needs recording, you may barely use FFmpeg at all.

Dashboard

Instead of Grafana, build your own.

Show things like:

 Live

Current viewers

Recording status

Duration

Bitrate

Recent recordings

This demonstrates your frontend and backend skills better than embedding Grafana.

Failover

For a project, don't overcomplicate it.

Use:

- Docker restart policies (`restart: unless-stopped` or `always`)
- Client auto-reconnect
- Health checks in your web app (optional)

This is enough to show resilience without building a clustered infrastructure.

Authentication

Supabase Auth gives you:

- Login
- Signup
- JWTs
- Password reset
- Google/GitHub OAuth (if you want)

No need for Keycloak or Authentik in a showcase project.

The only extra service I'd add

One small **Upload Worker**.

MediaMTX

↓

recordings/

↓

Upload Worker

↓

Supabase Storage

↓

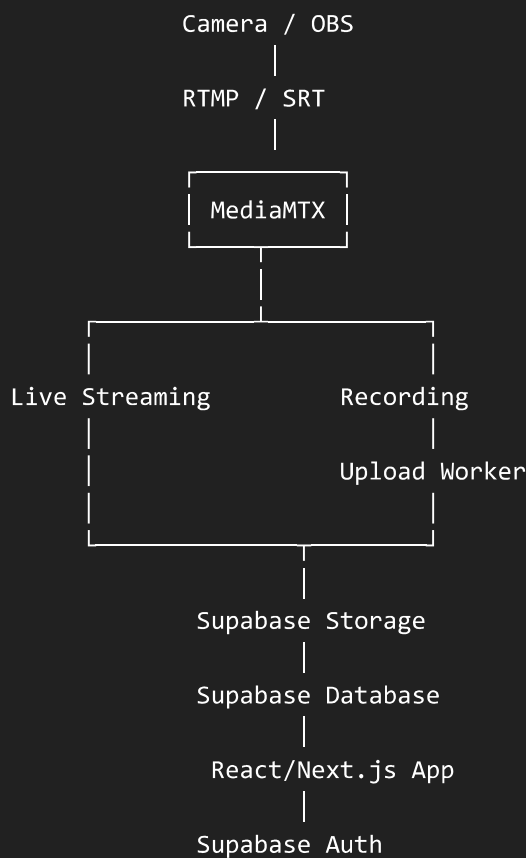
Supabase Database

Its only job is:

1. Detect when a recording finishes.
2. Upload it to Supabase Storage.
3. Save metadata (name, duration, size, URL) in the database.
4. Delete the local file if desired.

This can be a very small Node.js script.

Final Architecture



This is the stack I would commit to

- ☒ Live Streaming: MediaMTX
- ☒ Cloud Storage: Supabase Storage
- ☒ Recording: MediaMTX
- ☒ Encoding/Processing: FFmpeg (only when needed)
- ☒ Database: Supabase PostgreSQL
- ☒ Authentication: Supabase Auth
- ☒ Dashboard: Your own React/Next.js dashboard using the MediaMTX API and Supabase
- ☒ Monitoring: Built into your dashboard (stream status, recordings, viewers, uploads)

-  **Failover:** Docker restart policies + automatic client reconnect
-  **Deployment:** Docker Compose on a single VPS

For a portfolio or academic project, this strikes a good balance between simplicity and realism. It demonstrates the core concepts of a professional live streaming platform without requiring a complex multi-server production environment.